DIGITAL VLSI DESIGN · COURSEWORK REPORT

A 4-Bit Array Multiplier from RTL to GDSII

Design, verification, logic synthesis, timing analysis and place & route of an unsigned 4×4 multiplier on the SkyWater 130 nm process, using an open-source toolchain.

Shourya Sarkar — Roll No. 2023112013

guru shwadeep — Roll No. 2023112020

Team assignment (2 students)

15% of the total coursework

SKY130 · sky130_fd_sc_hd · LibreLane 3.0.16 · October 2026

Contents

1 Introduction and Design Overview

2 Part A — RTL Design and Verification

2.1 Simulation results and error-injection test

2.2 Worked example

2.3 Block diagram and waveform

3 Part B — Logic Synthesis with Yosys

4 Part C — Static Timing Analysis with OpenSTA

5 Part D — Physical Design: RTL to GDSII

6 Answers to the Report Questions

7 Conclusion

A Appendix A — Command Transcript

B Appendix B — Source Code Listings

1 Introduction and Design Overview

This report covers the design of a 4-bit unsigned array multiplier from RTL to a signed-off GDSII layout on the SkyWater 130 nm process. There are four stages: RTL design and exhaustive verification, logic synthesis with Yosys, static timing analysis with OpenSTA, and place and route with LibreLane. All four were run with open-source tools. The numbers in the tables come from the tool output of those runs; the small text files under `logs/` keep the original console transcripts, and Appendix A lists the commands in the order they were used.

The multiplier takes two 4-bit unsigned operands and produces an 8-bit product. Its combinational core is the standard array: sixteen AND gates form the partial products ppi[j] = a[j] · b[i], and twelve full adders, arranged as three rows of four, add the partial products one row at a time. Row 0 contains only AND gates and releases p[0] directly. The lowest sum bit of each adder row is a finished product bit (p[1], p[2], p[3]), and the last row also produces p[7:4]. Inputs are registered in a_reg and b_reg, and the result is registered in p, so the design has exactly 16 flip-flops, a latency of two clock cycles and a throughput of one result per cycle. The asynchronous active-low reset rst_n clears every register.

Table 1 lists the tools and the job each one does in the flow.

Stage	Tool (version used)	What it does here
RTL simulation	Icarus Verilog 13.0	Compiles rtl/ and tb/, runs the 256-vector testbench and writes mult.vcd
Logic synthesis	Yosys 0.62	Maps the RTL onto sky130_fd_sc_hd cells and writes the netlist and area report
Gate-level simulation	Icarus Verilog with the PDK cell models	Runs the same testbench on the synthesized netlist
Static timing	OpenSTA 2.7.0	Finds the critical path, worst slack, Tmin and Fmax on the tt corner
Place and route	LibreLane 3.0.16 (OpenROAD)	Floorplan, placement, CTS, routing, extraction, timing signoff and GDSII
Physical verification	Magic 8.3.623, Netgen 1.5.316, KLayout 0.30.7	DRC, LVS, and the layout views
PDK	SkyWater SKY130 (ciel 8afc8346)	Standard cells, technology files, Liberty models and KLayout layer properties

Table 1 — Tools and their role in the flow.

Parts B to D were run inside the LibreLane environment, which bundles the same tool versions as the IIC-OSIC-TOOLS image suggested in the assignment. The SKY130 PDK was installed with ciel. Part A was also re-run later with a local Icarus Verilog 12.0 and printed the same PASS and FAIL lines; those logs are in `logs/` as well.

Property	Value
Design	4-bit × 4-bit unsigned array multiplier, 8-bit product
Combinational core	16 AND gates + 12 full adders (3 rows of 4)
Registers	16 flip-flops: a_reg (4), b_reg (4), p (8); async reset rst_n
Latency	2 clock cycles (input registers, array, output register)
Throughput	1 result per clock cycle
Technology	SkyWater SKY130, sky130_fd_sc_hd
Die (Part D)	150 µm × 150 µm, FP_SIZING = absolute

Table 2 — Design facts.

2 Part A — RTL Design and Verification

rtl/full_adder.v is a 1-bit full adder written with two assign statements: sum = a ^ b ^ cin and cout = (a & b) | (b & cin) | (a & cin). rtl/mult_array.v instantiates twelve of these, four per adder row, and builds the sixteen partial-product AND gates from the registered operands. The internal wire names follow the table in Section 2 of the assignment: row 0 passes r0 = {1'b0, pp0[3:1]} down to row 1, each row adds its own partial product to the value from the row above, and the {carry, sums} bundle drops into the next row. The finished bits are collected as prod = {c3[4], s3[3:1], s3[0], s2[0], s1[0], pp0[0]} and registered into p. The multiplication operator is not used anywhere in the RTL.

tb/tb_mult.v generates a 10 ns clock and holds rst_n low for the first two cycles. It then applies all 256 combinations of a and b. For each vector it waits two rising edges: the first captures a and b into the input registers, the second captures the product into p. Only after that does it compare p with a * b; the * operator is allowed in the testbench. A mismatch prints one line, and the run ends with a single summary line. The testbench also dumps the full hierarchy to mult.vcd.

2.1 Simulation results and error-injection test

The clean run of the final RTL printed **PASS: 256 vectors checked, 0 errors**. For the error-injection test the sum equation in full_adder.v was changed to sum = a ^ b, which drops the carry-in. The testbench caught the fault in 95 of the 256 vectors, so the checker does detect real functional errors. The line was restored and the clean simulation was run again before moving on.

Run	Result
RTL simulation (vvp sim)	**PASS: 256 vectors checked, 0 errors**
Error injection, sum = a ^ b (vvp sim_err)	**FAIL: 256 vectors checked, 95 errors**; first mismatch a=3 b=3 → p=5, expected 9
Restored full_adder.v, re-run	**PASS: 256 vectors checked, 0 errors**

Table 3 — Part A simulation results.

The same two checks were repeated later with Icarus Verilog 12.0 and gave the same lines. The submitted mult.vcd is the dump from that RTL run, not from the gate-level simulation.

2.2 Worked example

The worked example uses A = (last digit of Member 1's roll number 2023112013) + 5 = 3 + 5 = 8 = 1000_2 and B = 7 = 0111_2. Since A is not 7, B stays 7 as the assignment allows. Only pp0, pp1 and pp2 contribute, because b[3] = 0 clears the third partial product. The shift-and-add calculation is:

	128	64	32	16	8	4	2	1
A	0	0	0	0	1	0	0	0
B	0	0	0	0	0	1	1	1
pp0 = A·b0	0	0	0	0	1	0	0	0
pp1 = A·b1, <<1	0	0	0	1	0	0	0	0
pp2 = A·b2, <<2	0	0	1	0	0	0	0	0
pp3 = A·b3, <<3	0	0	0	0	0	0	0	0
P = 00111000_2 = 56	0	0	1	1	1	0	0	0

Table 4 — Hand-worked 8 × 7 = 56 in binary, with the partial products shifted by their bit position.

The same calculation in the vertical layout used in Section 2 of the assignment:

```
      1 0 0 0      A = 1000  (8)
    x 0 1 1 1      B = 0111  (7)
    ---------
      1 0 0 0      pp0 = A AND b0 = 1000
    1 0 0 0        pp1 = A AND b1 = 1000, shifted left 1
  1 0 0 0          pp2 = A AND b2 = 1000, shifted left 2
0 0 0 0            pp3 = A AND b3 = 0000, shifted left 3
-------------
0 0 1 1 1 0 0 0    P = 00111000  (56)
```

2.3 Block diagram and waveform

Figure 1 shows the complete multiplier: the input registers, the diagonal array of sixteen AND gates, the three rows of four full adders with their carry chains, the product bus and the output register. Each gate is drawn at its product-bit column, so the figure also shows how p[0] to p[7] line up with the array.

Figure 2 is the example vector in the simulation waveform, rendered from mult.vcd. The testbench drives a = 1000 and b = 0111 at t = 2718 ns. The next rising edge, at 2725 ns, loads them into a_reg and b_reg; the edge after that, at 2735 ns, loads the product 00111000 = 0x38 = 56 into p. The time from applying the inputs to the product appearing is 17 ns: 7 ns to the first edge plus one 10 ns clock period between the two edges. The latency is two clock cycles, which is what the two marked edges show.

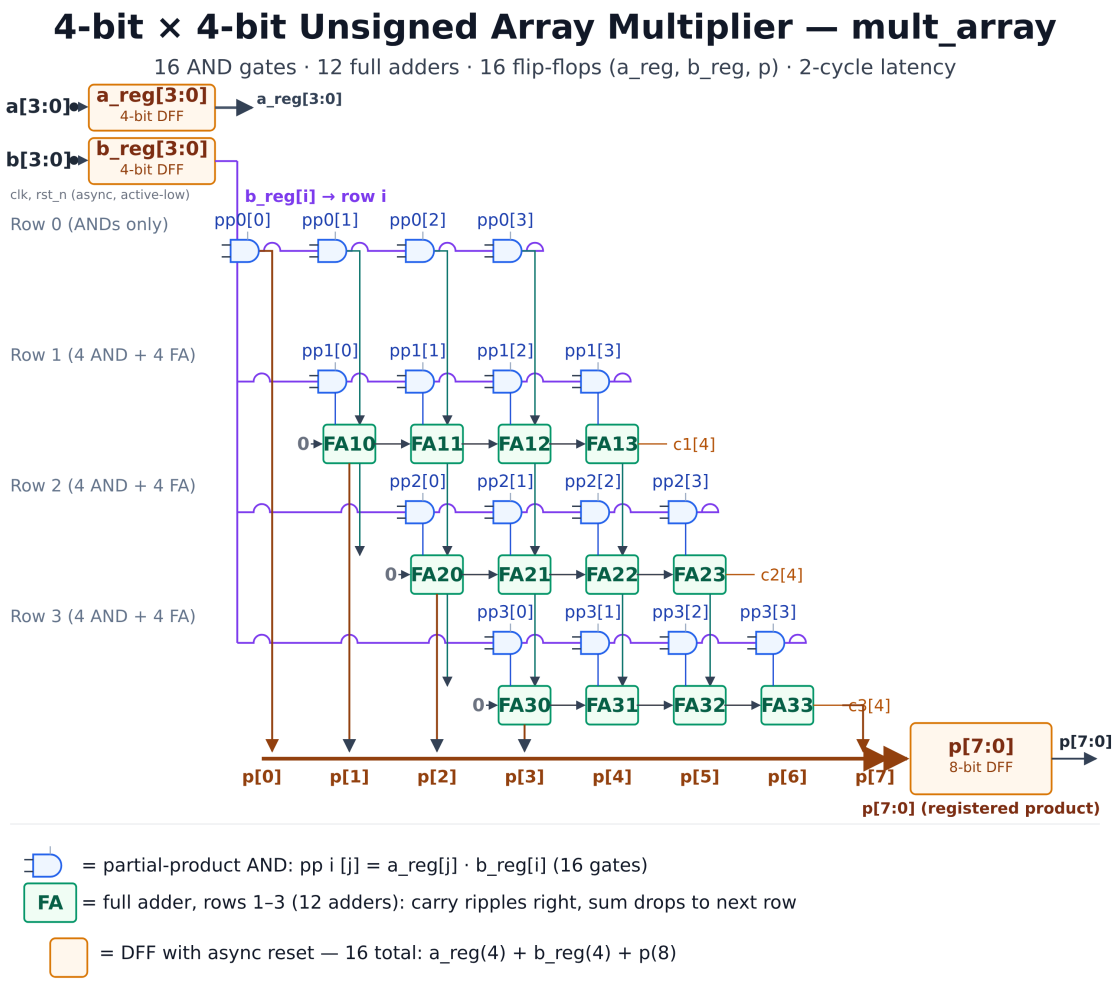


Figure 1 — Block diagram of mult_array: 16 AND gates, 12 full adders, input registers a_reg/b_reg and output register p. Blue is the partial product feeding an adder row, teal is a sum bit passed to the next row, amber is the ripple carry, and violet is the b_reg[i] row input.

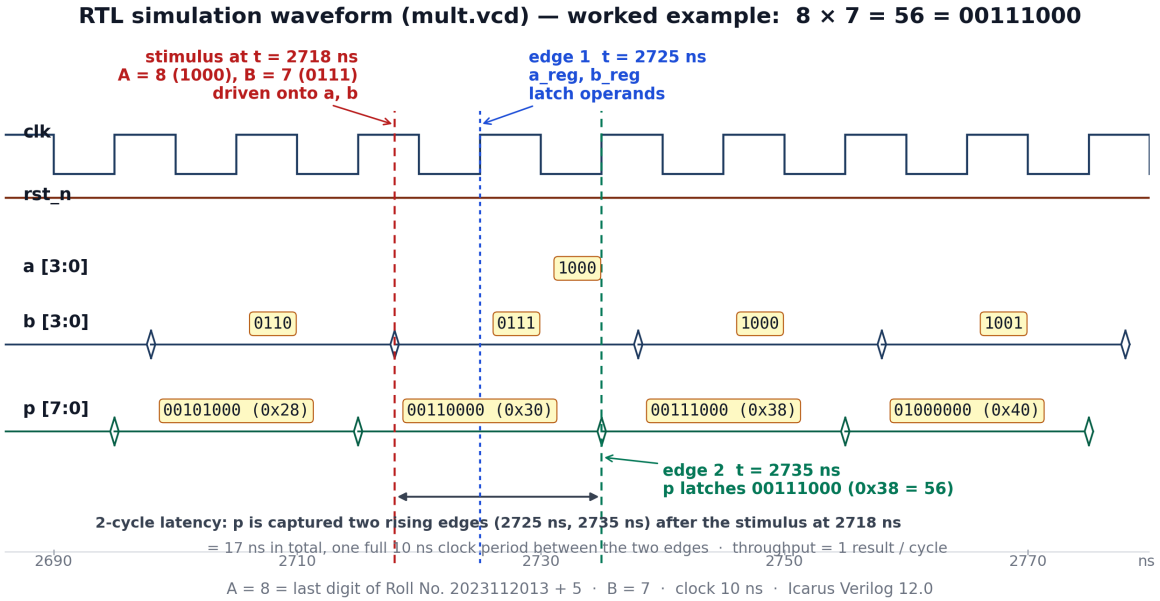


Figure 2 — RTL simulation waveform for A = 8, B = 7. The inputs are applied at 2718 ns; p[7:0] becomes 00111000 (56) at 2735 ns, two rising edges later.

3 Part B — Logic Synthesis with Yosys

synth/synth.tcl follows the seven steps in the assignment. It reads both RTL files, runs synth -top mult_array -flatten so the twelve full adders are merged into one module, maps the flip-flops with dfflibmap, maps the combinational logic with abc using the tt_025C_1v80 Liberty file, removes unused wires with opt_clean -purge, writes the area report with stat, and writes the netlist with write_verilog. Yosys maps the asynchronous active-low reset to dfrtp cells.

The result is 65 cells: 49 combinational cells and 16 flip-flops. The three most-used types are dfrtp_1 (16), nand2_1 (10) and xnor2_1 (7). The carry logic maps mostly to NAND and AOI/OAI structures, the sum bits use XOR and XNOR gates, and two maj3 majority gates appear on the deeper carry chain. The total cell area is 751.971 µm², of which the flip-flops take 400.384 µm² (53.24 %).

Metric	Value
Total number of cells	65
Number of flip-flops	16 (sky130_fd_sc_hd__dfrtp_1)
Total cell area	751.971 µm²
Flip-flop area	400.384 µm² (16 × 25.024 µm²)
Flip-flop area (% of total)	53.24 %
Three most-used cell types	dfrtp_1 × 16, nand2_1 × 10, xnor2_1 × 7
Netlist simulation result	**PASS: 256 vectors checked, 0 errors**

Table 5 — Part B results, from synth/mult_array_stat.rpt.

The gate-level netlist was simulated with the SkyWater cell models in FUNCTIONAL mode with unit delays, using the command from the assignment. It passes all 256 vectors: **PASS: 256 vectors checked, 0 errors**. Synthesis therefore did not change the behaviour of the design.

4 Part C — Static Timing Analysis with OpenSTA

OpenSTA checks every path in the netlist against the clock period without simulating it. sta/sta.tcl reads the tt_025C_1v80 Liberty library and the synthesized netlist, links the design, creates the clock on port clk, sets 1 ns input and output delays on the data ports, prints the worst setup slack and writes the full path report. It was run at T = 10 ns and again at the calculated Tmin.

At T = 10 ns the worst setup slack is +7.35 ns. The minimum period is therefore Tmin = 10 − 7.35 = 2.65 ns, which gives Fmax = 1000 / 2.65 = 377.4 MHz. The second run at T = 2.65 ns reports 0.00 ns, so the number is consistent. At full speed the two-cycle latency is 2 × 2.65 = 5.30 ns, and the throughput is one result per cycle, or 377.4 million results per second.

Metric	Value
Worst slack at T = 10 ns	+7.35 ns (MET)
Tmin	2.65 ns
Fmax	377.4 MHz
Slack at T = Tmin	0.00 ns
Critical path start point	a_reg[1] flip-flop (cell _099_, dfrtp_1)
Critical path end point	p[6] flip-flop (cell _096_, D = fa33.sum)
Cells on the critical path	10 (8 combinational + 2 flip-flops)
Latency (2 × Tmin)	5.30 ns
Throughput	377.4 million results per second

Table 6 — Part C results, from sta/mult_array_timing.rpt.

The critical path is from the a_reg[1] flip-flop (_099_) to the p[6] flip-flop (_096_). It goes through the AND gate that forms pp1[1] (_041_), seven compound gates produced by flattening and re-optimising the adder rows, and the final XNOR that produces fa33.sum (_089_). Because the netlist is flattened, the cells no longer correspond one to one with the RTL full adders; Table 7 lists each one by the function its connections actually perform. The path has 8 combinational cells and 2 flip-flops, and the data arrives at 2.525 ns.

The assignment estimates about 2N = 8 full-adder delays. The measured path has 8 mapped gate stages, but those stages do not map one-to-one onto full adders: synthesis turns each adder's sum and carry into one or two library gates, and the first adder of each row becomes a half adder because its carry-in is tied to zero. In full-adder terms the path is closer to five adder traversals, so the estimate and the measurement are in the same range once the units are compared properly.

#	Instance / cell	Function in the array	Delay (ns)
1	_099_ / dfrtp_1	a_reg[1] flip-flop, startpoint; clk→Q delay	0.387
2	_041_ / and2_0	pp1[1] = a_reg[1]·b_reg[1]; output _000_	0.201
3	_051_ / o21ai_0	O21AI Y = ¬((A1|A2)·B1) combining _000_, pp0[2] (_004_) and _007_	0.209
4	_052_ / a21o_1	AND-OR X = (A1·A2)|B1 over _006_, _008_ and ¬pp2[0] (_003_); output _009_	0.169
5	_060_ / xor3_1	XOR3 of _009_, _012_ (¬pp2[1]) and _015_; output _016_ is s2[1] complemented	0.419
6	_061_ / lpflow_inputiso1p_1	Input-isolation cell used as OR: _017_ = ¬pp3[0] | ¬s2[1]	0.249
7	_081_ / maj3_1	Majority of _017_, ¬pp3[1] (_018_) and _028_; output _035_	0.377
8	_084_ / maj3_1	Majority of ¬pp3[2] (_029_), _033_ and _035_; output _037_	0.382
9	_089_ / xnor2_1	XNOR of _037_ and _040_, producing fa33.sum = s3[3] = p[6]	0.131
10	_096_ / dfrtp_1	p[6] flip-flop, endpoint; library setup time	−0.125

Table 7 — Cells on the critical path, in order (tt corner).

5 Part D — Physical Design: RTL to GDSII

LibreLane runs the whole physical flow: synthesis, floorplan on the fixed 150 µm × 150 µm die, tap-cell insertion, the power grid, placement, clock-tree synthesis, routing, RC extraction, multi-corner timing signoff, DRC with Magic and KLayout, LVS with Netgen, and GDSII stream-out. pnr/config.json contains the six settings the assignment asks for. The clock period starts from the Part C result: 1.2 × 2.65 = 3.18 ns, rounded up to the next 0.5 ns, which is 3.5 ns.

The first run finished with 0 DRC and 0 LVS errors, but LibreLane 3 checks nine timing corners and the slow-slow corner (ss, 100 °C, 1.60 V) still failed setup. Following the assignment rule, the period was increased by 0.5 ns and the flow was run again. Eight runs were needed. The worst-corner slack improved from −1.801 ns at 3.5 ns to −0.044 ns at 6.0 ns, moved back to −0.128 ns at 6.5 ns because the resizer picked a different buffering strategy, and finally closed at +0.241 ns with no violations at 7.0 ns.

Run	CLOCK_PERIOD (ns)	Worst setup slack (ns)	Setup violations	Result
1	3.5	−1.801	15	timing fail
2	4.0	−1.319	14	timing fail
3	4.5	−0.819	12	timing fail
4	5.0	−0.319	12	timing fail
5	5.5	−0.071	5	timing fail
6	6.0	−0.044	2	timing fail
7	6.5	−0.128	4	timing fail
8	7.0	+0.241	0	pass

Table 8 — Part D timing-closure runs.

The final run has 0 routing DRC errors, 0 Magic DRC errors, 0 KLayout DRC errors and 0 LVS errors. The antenna check passes, hold timing is met in every corner, and the worst setup slack is +0.241 ns. Two kinds of warnings remain. There are 11 max-slew (transition) warnings in the three slow-slow corner variants; all tt and ff corners are clean. The flow's resizer inserted three buffers to fix the one slew violation it found during the run, but it did not clear all of them at ss. There are also 446 lint warnings, all from the Verilator.Lint step. Each one is a TIMESCALEMOD warning about an auto-generated PDK black-box stub that has no timescale directive while rtl/full_adder.v does. They are notices about the generated PDK files, not about the design; the lint error count is 0.

Metric	Value
CLOCK_PERIOD used	7.0 ns
Number of runs	8 (3.5 ns to 7.0 ns)
Core area	17 759.5 µm² (die 150 × 150 µm = 22 500 µm²); std-cell utilisation 8.9 %
Cells after synthesis	65
Cells after place and route	4 821 = 346 standard cells (including 245 tap cells) + 4 475 fill/decap cells
Kinds of cells added	decap_3, fill_1/fill_2, tapvpwrvgnd_1, clkdlybuf4s25_1, clkbuf_16, inv_2
Wirelength	3 030 µm of detailed-routing wire, 701 single-cut vias
Setup slack	+0.241 ns worst corner; tt +3.438 ns worst / +3.463 ns nominal
Post-route Fmax	148.0 MHz worst corner; 280.7 MHz worst tt / 282.7 MHz nominal tt
Fmax from Part C	377.4 MHz
DRC errors	0 (router, Magic and KLayout)
LVS errors	0 (Netgen; no unmatched devices, nets or pins)

Table 9 — Part D results, from pnr/metrics.json at CLOCK_PERIOD = 7.0 ns.

Table 10 lists the cells that place and route added. The fill and decap cells occupy the spare sites in the rows, the tap cells protect against latch-up, the CTS buffers balance the clock, and the resizer buffers repair slew and fanout on long nets. The sixteen flip-flops were upsized to dfrtp_2, and the combinational logic was re-mapped to the _2 drive strength.

Cell (sky130_fd_sc_hd__)	Count	Why it is there
decap_3	4 197	Decoupling capacitance in the spare sites of the cell rows
fill_1	215	Filler that keeps the N-well and implant continuous
fill_2	63	Same as fill_1, for wider gaps
tapvpwrvgnd_1	245	Substrate and well taps at the required intervals
clkdlybuf4s25_1	20	Resizer buffers for slew, fanout and setup repair
clkbuf_16	3	Clock-tree buffers
inv_2	2	Logic re-buffering during placement
dfrtp_2	16	The sixteen flip-flops, upsized by the resizer
60 combinational gates (_2 drive)	60	The multiplier logic, re-mapped and upsized
Total	4 821	346 standard cells including taps, plus 4 475 fill/decap

Table 10 — Cell types in the final netlist.

The submitted metrics.json and mult_array.gds are copies of the final folder of the successful run, runs/RUN_2026-10-22_17-13-53. A repeat run at the same period (runs/RUN_2026-10-22_19-17-31) produced the same instance count, areas, utilisation, wirelength, slack in all nine corners, DRC and LVS counts, and the same slow-corner slew warnings. The regenerated GDS differs only in its embedded timestamp.

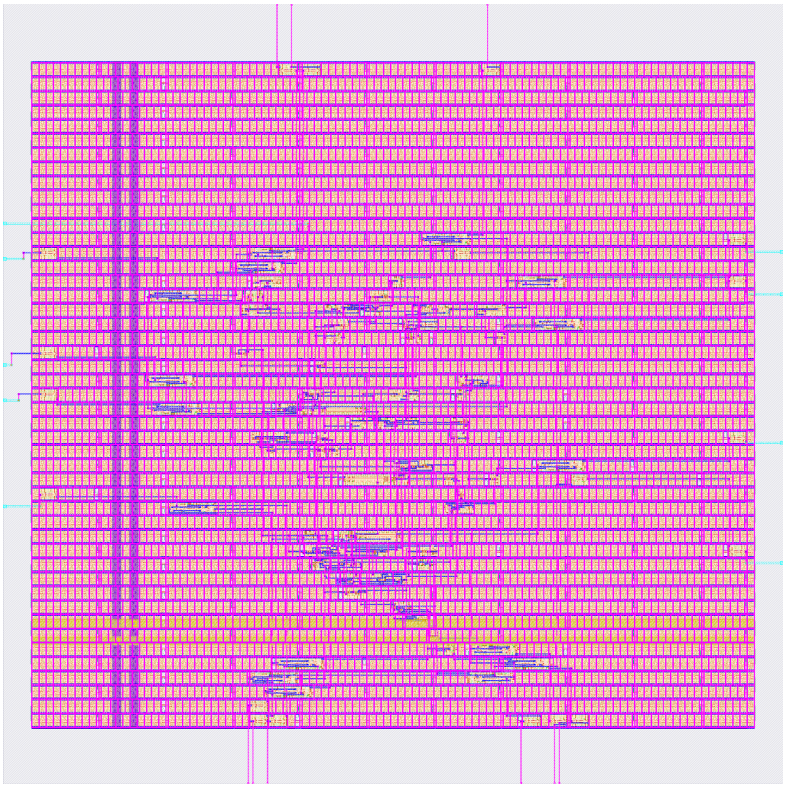


Figure 3 — The final 150 µm × 150 µm die in KLayout, with the SKY130A layer colours. The cells and routing occupy the central core and the pin ring sits on the die edge.

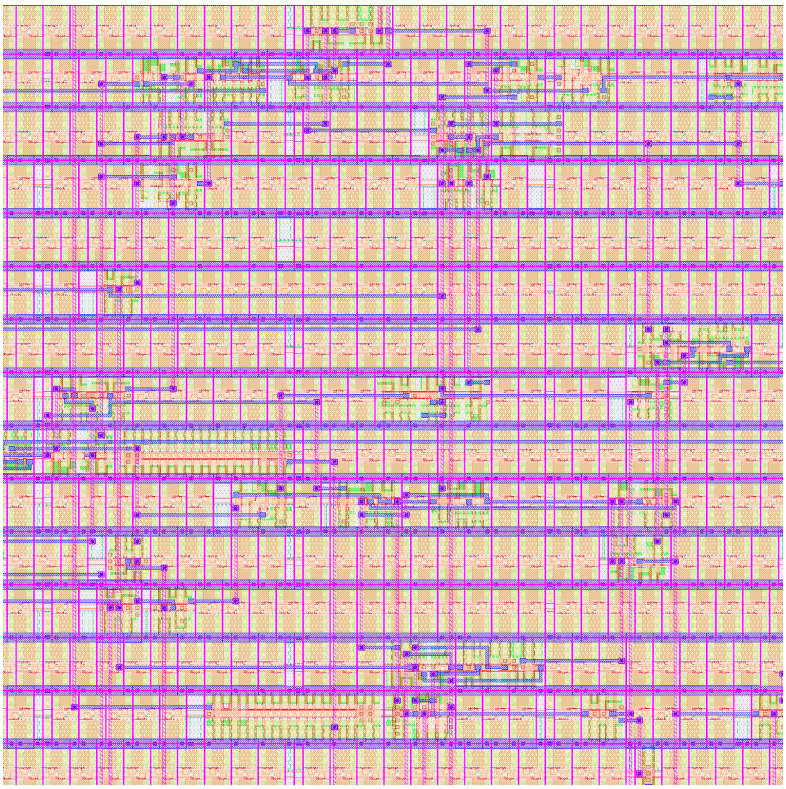


Figure 4 — A 40 µm × 40 µm zoom into the densest part of the core: standard-cell rows with tap and fill/decap cells, and the met1/met2 routing above them.

6 Answers to the Report Questions

Q1. Why the logic cell count is not simply 16 + 12.

Synthesis produced 65 cells, 16 of which are flip-flops, so the combinational array mapped to 49 cells rather than the 28 suggested by the structural count. The two numbers count different things: 16 + 12 is the number of RTL blocks, while synthesis maps Boolean equations onto library primitives. A full adder is two Boolean functions, sum and carry, and a naive mapping needs 5 to 9 gates for the pair. Yosys and ABC then optimise that. Constant propagation removes the tied-zero carry-ins (c1[0] = c2[0] = c3[0] = 0), so the first adder of each row becomes a half adder. Boolean optimisation rewrites the carry as a majority function, which the library provides directly as maj3. Logic sharing folds partial-product ANDs into compound AND-OR, AOI and OAI gates such as a21o, a22oi and o2111ai, and structural hashing reduces p[0] = a_reg[0]·b_reg[0] to a single and2. The 49-cell result comes from that technology mapping, before any physical-design step.

Q2. The critical path.

The path is register to register: a_reg[1] (_099_) → pp1[1] AND (_041_) → seven compound gates that span the flattened rows 1 to 3 (the O21AI _051_, the AND-OR _052_, the XOR3 _060_, the input-isolation cell _061_ used as an OR, and the majority gates _081_ and _084_) → the row-3 sum s3[3] from the XNOR _089_ (fa33.sum) → p[6] (_096_). That is 8 combinational cells and 2 flip-flops, as listed in Table 7. The flattened netlist has no discrete adders left, so the cells are described by the connections they actually have. The assignment's estimate is about 2N = 8 full-adder delays. The measured path is 2.525 ns over 8 mapped gate stages, which is roughly five adder traversals. The difference comes from the half adders at the start of each row and from the XOR sum stages being faster than a full carry ripple. The hand estimate counts adder traversals and the measured path counts library gates, so the two agree within the accuracy of the model.

Q3. Flip-flop area share.

The flip-flops take 400.384 µm² out of 751.971 µm², or 53.24 %. A dfrtp cell is large for this library, 25.024 µm², because it contains a master-slave latch pair, an asynchronous reset and a buffer chain. There are only 49 small logic cells next to the 16 flip-flops, since 4-bit multiplication compresses well: the partial products are single AND terms and the adder rows share most of their structure. In a wider datapath the logic would grow and the register share would fall, but for a design this small the registers are a large fixed cost.

Q4. Post-route Fmax versus Part C.

Part C predicted 377.4 MHz. After place and route the design closes at 7.0 ns with +0.241 ns worst-corner slack, so the worst-corner Fmax is 148 MHz. On the worst tt corner it is 280.7 MHz and on the nominal tt corner, which is what Part C analysed, it is 282.7 MHz. There are three reasons for the gap. First, Part C used an ideal clock and no interconnect; post-route timing includes extracted wire RC, and the load on many nets roughly doubles their delay. Second, the clock tree adds skew and the flow applies 0.25 ns of clock uncertainty. Third, LibreLane signs off in nine corners, and the 100 °C / 1.60 V slow-slow corner is about 2 to 2.5 times slower than tt, so the 7.0 ns period is really an ss-corner guarantee. The added cells each have a job: taps protect against latch-up, fill and decap keep the wells continuous and steady the supply, repair buffers restore slew and fanout, and CTS buffers balance the clock. The 11 slow-corner max-slew warnings are a signal-integrity quality note; they are not part of the assignment's DRC, LVS and slack criteria.

Q5. DRC and LVS.

DRC checks that the geometry follows the process rules: minimum widths, spacings, enclosures and via sizes, so the masks can be printed. LVS checks that the netlist extracted from the layout, with its transistors and connections, matches the source netlist, so the silicon implements the verified circuit with no shorts, opens or wrong devices. Both are needed because they catch different failures. A layout can be DRC clean and still contain one shorted net that changes the circuit, which LVS would catch. It can also match the netlist and still have a spacing violation that kills yield, which DRC would catch. In this project both Magic and KLayout report 0 DRC errors, and Netgen reports 0 LVS errors with no unmatched devices, nets or pins.

Q6. Scaling to 8 bits.

Using the formulas from Section 2 with N = 8, the AND count grows as N^2 from 16 to 64, and the full-adder count grows as $N(N-1)$ from 12 to $8 \times 7 = 56$, a factor of 4.7. The critical path grows as roughly 2N full-adder delays, from about 8 units for the 4-bit array to about 16 units for the 8-bit one. Keeping the units consistent and applying a simple linear scaling to the measured 2.65 ns, the 8-bit Tmin is about $2 \times 2.65 = 5.30$ ns, so Fmax is about $1000 / 5.30 = 189$ MHz, roughly half of 377.4 MHz. This is a paper estimate, not an 8-bit synthesis or timing run. The register count would grow from 16 to 8 + 8 + 16 = 32 flip-flops, the latency would stay at two cycles (about 10.6 ns at the estimated Tmin), and the throughput would still be one result per cycle.

7 Conclusion

The full open-source flow produced what the assignment asked for: an exhaustively verified 4-bit array multiplier (256 of 256 vectors on both RTL and gate level, with the error-injection test confirming that the checker works), a 65-cell SKY130 netlist with an area of 751.971 μm^2, a single-corner Fmax of 377 MHz from OpenSTA, and a signed-off 150 µm × 150 µm GDSII with 0 DRC and 0 LVS errors after eight timing-closure runs at 7.0 ns. The 11 slow-corner max-slew warnings and the 446 PDK-stub lint warnings are the two quality notes. The main lesson is the size of the gap between a pre-layout estimate and a multi-corner post-route result. An ideal-clock, typical-corner analysis makes the design look more than twice as fast as it is after extraction, and the physical cells exist to make the layout manufacturable and the timing real.

The waveform in Figure 2 is rendered from the mult.vcd of the passing RTL simulation. The block diagram is produced by scripts/block_diagram.py, and the two layout views were rendered in KLayout from the final mult_array.gds with the SKY130A layer properties. Appendix A lists the commands, and the README in the repository has the same list, so the results can be regenerated from the submitted files.

Appendix A — Command Transcript

The commands below are in execution order. Parts B to D were run in the LibreLane environment with the SKY130 PDK; PDK_ROOT pointed at the sky130A installation. Unless noted, run them from the repository root.

Environment

```
iverilog -V          # Icarus Verilog 13.0 (devel)
yosys -V             # Yosys 0.62 (git 7326bb7)
sta -version         # OpenSTA 2.7.0
librelane --version  # LibreLane v3.0.16
klayout -v           # KLayout 0.30.7  (Magic 8.3.623, Netgen 1.5.316)
echo $PDK_ROOT       # SKY130 PDK root (ciel, version 8afc8346)
```

Part A — RTL simulation and error injection

```
iverilog -g2012 -o sim tb/tb_mult.v rtl/full_adder.v rtl/mult_array.v
vvp sim
# -> PASS: 256 vectors checked, 0 errors   (mult.vcd written)
gtkwave mult.vcd

# full_adder.v: temporarily change sum = a ^ b ^ cin to sum = a ^ b
iverilog -g2012 -o sim_err tb/tb_mult.v rtl/full_adder.v rtl/mult_array.v && vvp sim_err
# -> FAIL: 256 vectors checked, 95 errors  (first: a=3 b=3 -> p=5, expected 9)
# restore the line, then:
vvp sim
# -> PASS: 256 vectors checked, 0 errors
```

Part B — synthesis and gate-level simulation

```
yosys -c synth/synth.tcl
# -> synth/mult_array_netlist.v + synth/mult_array_stat.rpt
# -> 65 cells, 16 dfrtp_1, area 751.971 um2 (53.24 % sequential)

CELLS=$PDK_ROOT/sky130A/libs.ref/sky130_fd_sc_hd/verilog
iverilog -g2012 -DFUNCTIONAL -DUNIT_DELAY=#1 -o gls \
     tb/tb_mult.v $CELLS/primitives.v $CELLS/sky130_fd_sc_hd.v \
     synth/mult_array_netlist.v
vvp gls
# -> PASS: 256 vectors checked, 0 errors
```

Part C — static timing analysis

```
sta sta/sta.tcl                  # T = 10 ns
# -> worst slack max 7.35 ns    Tmin = 2.65 ns, Fmax = 377.4 MHz
STA_PERIOD=2.65 sta sta/sta.tcl # confirmation run at Tmin
# -> worst slack max 0.00 ns
```

Part D — LibreLane runs

```
cd pnr
librelane --pdk-root $PDK_ROOT config.json   # 3.5 ns -> slack -1.801  fail
# increase CLOCK_PERIOD by 0.5 ns and repeat:
librelane --pdk-root $PDK_ROOT config.json   # 4.0 -> -1.319  fail
librelane --pdk-root $PDK_ROOT config.json   # 4.5 -> -0.819  fail
librelane --pdk-root $PDK_ROOT config.json   # 5.0 -> -0.319  fail
librelane --pdk-root $PDK_ROOT config.json   # 5.5 -> -0.071  fail
librelane --pdk-root $PDK_ROOT config.json   # 6.0 -> -0.044  fail
librelane --pdk-root $PDK_ROOT config.json   # 6.5 -> -0.128  fail
librelane --pdk-root $PDK_ROOT config.json   # 7.0 -> +0.241  pass
# final run: runs/RUN_2026-10-22_17-13-53
cp runs/RUN_2026-10-22_17-13-53/final/metrics.json .
cp runs/RUN_2026-10-22_17-13-53/final/gds/mult_array.gds .
```

Figures

```
python3 scripts/render_waveform.py    # screenshots/waveform_example.png
python3 scripts/block_diagram.py      # screenshots/block_diagram.png
QT_QPA_PLATFORM=offscreen klayout -b -r scripts/klayout_shots.py
# -> screenshots/mult_array_die.png + mult_array_zoom.png
```

Appendix B — Source Code Listings

The six files submitted in the repository, as they appear under rtl/, tb/, synth/, sta/ and pnr/.

B.1 rtl/full_adder.v

```
//------------------------------------------------------------------------------
// full_adder.v
// 1-bit full adder, combinational.
//
// Ports (as required by the assignment, Section 5):
//   a, b, cin  : inputs
//   sum, cout  : outputs
//
// Written with assign statements only (Part A requirement).
//------------------------------------------------------------------------------
`timescale 1ns/1ps

module full_adder (
    input  wire a,
    input  wire b,
    input  wire cin,
    output wire sum,
    output wire cout
);

    // sum = XOR of the three inputs
    assign sum  = a ^ b ^ cin;

    // carry-out = majority function of the three inputs
    assign cout = (a & b) | (b & cin) | (a & cin);

endmodule
```

B.2 rtl/mult_array.v

```
//------------------------------------------------------------------------------
// mult_array.v
// 4-bit x 4-bit unsigned array multiplier, 8-bit product (Part A).
//
// Ports (Section 5 of the assignment):
//   clk   : input, 1 bit, rising-edge clock
//   rst_n : input, 1 bit, asynchronous reset, active low (clears all registers)
//   a     : input, 4 bits, multiplicand
//   b     : input, 4 bits, multiplier
//   p     : output, 8 bits, registered product a * b
//
// Structure (Section 2):
//   Row 0 : 4 AND gates (pp0 = a & b[0]).
//           p[0] = pp0[0]; the remaining bits pass down as
//           r0 = {1'b0, pp0[3], pp0[2], pp0[1]}.
//   Rows 1..3 : 4 AND gates + a chain of 4 full adders each (12 FAs total).
//           Row i adds partial product ppi to the value ri-1 passed down
//           from row i-1. The lowest sum bit of each row is a finished
//           product bit (p[1], p[2], p[3]); the top bits plus the final
//           carry become p[7:4].
//
// Registers: a_reg, b_reg (4+4 bits) and p (8 bits) = 16 flip-flops.
// Latency: 2 clock cycles (input regs -> combinational array -> output reg).
// Throughput: 1 result per cycle.
//
// The "*" operator is NOT used here (it is only allowed in the testbench).
// The array is built from 12 explicit instances of full_adder.
//------------------------------------------------------------------------------
`timescale 1ns/1ps

module mult_array (
    input  wire       clk,
    input  wire       rst_n,
    input  wire [3:0] a,
    input  wire [3:0] b,
    output reg  [7:0] p
);

    //--------------------------------------------------------------------
    // Input registers (asynchronous reset, active low)
    //--------------------------------------------------------------------
    reg [3:0] a_reg, b_reg;

    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            a_reg <= 4'd0;
            b_reg <= 4'd0;
        end else begin
            a_reg <= a;
            b_reg <= b;
        end
    end

    //--------------------------------------------------------------------
    // Partial products: ppi[j] = a_reg[j] AND b_reg[i]  (16 AND gates)
    //--------------------------------------------------------------------
    wire [3:0] pp0, pp1, pp2, pp3;

    assign pp0 = a_reg & {4{b_reg[0]}};
    assign pp1 = a_reg & {4{b_reg[1]}};
    assign pp2 = a_reg & {4{b_reg[2]}};
    assign pp3 = a_reg & {4{b_reg[3]}};

    //--------------------------------------------------------------------
    // Row 0: AND gates only.
    //   p[0] = pp0[0] is finished; the remaining 3 bits pass down,
    //   shifted one column left: r0 = {0, pp0[3], pp0[2], pp0[1]}.
    //--------------------------------------------------------------------
    wire [3:0] r0;
    assign r0 = {1'b0, pp0[3:1]};

    //--------------------------------------------------------------------
    // Row 1: full adders fa10..fa13
    //   FA j adds pp1[j], r0[j] and carry c1[j]; c1[0] = 0.
    //   p[1] = s1[0]; r1 = {c1[4], s1[3], s1[2], s1[1]} passes down.
    //--------------------------------------------------------------------
    wire [3:0] s1;
    wire [4:0] c1;
```

```
    full_adder fa10 (pp1[0], r0[0], c1[0], s1[0], c1[1]);
    full_adder fa11 (pp1[1], r0[1], c1[1], s1[1], c1[2]);
    full_adder fa12 (pp1[2], r0[2], c1[2], s1[2], c1[3]);
    full_adder fa13 (pp1[3], r0[3], c1[3], s1[3], c1[4]);

    assign c1[0] = 1'b0;
    wire [3:0] r1;
    assign r1 = {c1[4], s1[3:1]};

    //---------------------------------------------------------------------
    // Row 2: full adders fa20..fa23 (same pattern, inputs pp2 and r1)
    //---------------------------------------------------------------------
    wire [3:0] s2;
    wire [4:0] c2;

    full_adder fa20 (pp2[0], r1[0], c2[0], s2[0], c2[1]);
    full_adder fa21 (pp2[1], r1[1], c2[1], s2[1], c2[2]);
    full_adder fa22 (pp2[2], r1[2], c2[2], s2[2], c2[3]);
    full_adder fa23 (pp2[3], r1[3], c2[3], s2[3], c2[4]);

    assign c2[0] = 1'b0;
    wire [3:0] r2;
    assign r2 = {c2[4], s2[3:1]};

    //---------------------------------------------------------------------
    // Row 3: full adders fa30..fa33 (inputs pp3 and r2).
    //   p[3] = s3[0]; p[7:4] = {c3[4], s3[3], s3[2], s3[1]}.
    //---------------------------------------------------------------------
    wire [3:0] s3;
    wire [4:0] c3;

    full_adder fa30 (pp3[0], r2[0], c3[0], s3[0], c3[1]);
    full_adder fa31 (pp3[1], r2[1], c3[1], s3[1], c3[2]);
    full_adder fa32 (pp3[2], r2[2], c3[2], s3[2], c3[3]);
    full_adder fa33 (pp3[3], r2[3], c3[3], s3[3], c3[4]);

    assign c3[0] = 1'b0;

    //---------------------------------------------------------------------
    // Combinational product (collect the finished bits)
    //   prod[0]=pp0[0], prod[1]=s1[0], prod[2]=s2[0], prod[3]=s3[0],
    //   prod[7:4] = {c3[4], s3[3:1]}
    //---------------------------------------------------------------------
    wire [7:0] prod;
    assign prod = {c3[4], s3[3:1], s3[0], s2[0], s1[0], pp0[0]};

    //---------------------------------------------------------------------
    // Output register (asynchronous reset, active low)
    //---------------------------------------------------------------------
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            p <= 8'd0;
        end else begin
            p <= prod;
        end
    end

endmodule
```

B.3 tb/tb_mult.v

```
//------------------------------------------------------------------------------
// tb_mult.v
// Exhaustive, self-checking testbench for mult_array (Part A).
//
// Requirements implemented (assignment, Part A):
//   * clock with a 10 ns period;
//   * rst_n held low for the first 2 clock cycles;
//   * applies all 256 input combinations (a = 0..15, b = 0..15);
//   * for each combination waits for the 2-cycle latency, then compares
//     p with the expected value a * b ("*" is allowed in the testbench only);
//   * prints a line for every mismatch;
//   * at the end prints exactly one summary line:
//         PASS: 256 vectors checked, 0 errors
//     or  FAIL: 256 vectors checked, <n> errors
//   * writes the waveform file mult.vcd.
//------------------------------------------------------------------------------
`timescale 1ns/1ps

module tb_mult;

    reg        clk;
    reg        rst_n;
    reg  [3:0] a;
    reg  [3:0] b;
    wire [7:0] p;

    integer    i, j;
    integer    errors;

    //--------------------------------------------------------------------
    // Device under test
    //--------------------------------------------------------------------
    mult_array dut (
        .clk   (clk),
        .rst_n (rst_n),
        .a     (a),
        .b     (b),
        .p     (p)
    );

    //--------------------------------------------------------------------
    // Clock: 10 ns period (toggle every 5 ns)
    //--------------------------------------------------------------------
    initial clk = 1'b0;
    always #5 clk = ~clk;

    //--------------------------------------------------------------------
    // Stimulus + checking
    //--------------------------------------------------------------------
    initial begin
        // waveform dump
        $dumpfile("mult.vcd");
        $dumpvars(0, tb_mult);

        errors = 0;
        rst_n  = 1'b0;      // reset asserted
        a      = 4'd0;
        b      = 4'd0;

        // hold rst_n low for the first 2 clock cycles
        @(posedge clk);
        @(posedge clk);
        #1  rst_n = 1'b1;   // release reset just after the 2nd edge

        // apply all 256 input combinations
        for (i = 0; i < 16; i = i + 1) begin
            for (j = 0; j < 16; j = j + 1) begin
                a = i[3:0];
                b = j[3:0];

                // 2-cycle latency: edge 1 captures a/b into the input
                // registers, edge 2 captures the product into p
                @(posedge clk);
                @(posedge clk);
                #3;  // let outputs settle (also safe for unit-delay GLS)

                if (p !== (a * b)) begin
                    errors = errors + 1;
```

```
                    $display("MISMATCH: a=%0d b=%0d -> p=%0d, expected %0d",
                             a, b, p, a * b);
                end
            end
        end

        // exactly one summary line
        if (errors == 0)
            $display("PASS: 256 vectors checked, 0 errors");
        else
            $display("FAIL: 256 vectors checked, %0d errors", errors);

        $finish;
    end

endmodule
```

B.4 synth/synth.tcl

```
#-------------------------------------------------------------------------------
# synth.tcl — Yosys synthesis script (Part B)
#
# Maps the array multiplier RTL onto SkyWater SKY130 (sky130_fd_sc_hd)
# standard cells.
#
# Run from the repository root:
#     yosys -c synth/synth.tcl
#
# Requires the SKY130 PDK, e.g. installed by ciel/volare with
# PDK_ROOT pointing at the PDK root directory (as in IIC-OSIC-TOOLS,
# where $PDK_ROOT is /foss/pdks and $PDK is sky130A).
#-------------------------------------------------------------------------------

# Liberty timing library, typical corner
set lib $::env(PDK_ROOT)/sky130A/libs.ref/sky130_fd_sc_hd/lib/sky130_fd_sc_hd__tt_025C_1v80.lib

# 1. Read the design (both files: mult_array instantiates full_adder)
yosys read_verilog rtl/full_adder.v rtl/mult_array.v

# 2. Generic synthesis; flatten the full_adder cells into one module
yosys synth -top mult_array -flatten

# 3. Map flip-flops to SKY130 flip-flop cells (dfrtp: async reset, active low)
yosys dfflibmap -liberty $lib

# 4. Map the remaining logic to SKY130 gate cells
yosys abc -liberty $lib

# 5. Remove unused wires / cells
yosys opt_clean -purge

# 6. Write the cell and area report
yosys tee -o synth/mult_array_stat.rpt stat -liberty $lib

# 7. Write the gate-level netlist
yosys write_verilog -noattr synth/mult_array_netlist.v
```

B.5 sta/sta.tcl

```
#-------------------------------------------------------------------------------
# sta.tcl — OpenSTA static timing analysis script (Part C)
#
# Analyses the synthesized netlist against the clock period T, typical
# corner only (tt_025C_1v80).
#
# Run from the repository root:
#     sta sta/sta.tcl                       (T = 10 ns, first run)
#     STA_PERIOD=<Tmin> sta sta/sta.tcl     (second run at T = Tmin,
#                                             to confirm Fmax)
#
# Requires PDK_ROOT pointing at the SKY130 PDK root.
#-------------------------------------------------------------------------------

# Liberty timing library, typical corner
set lib $::env(PDK_ROOT)/sky130A/libs.ref/sky130_fd_sc_hd/lib/sky130_fd_sc_hd__tt_025C_1v80.lib

# Clock period T (ns). Default 10 ns (first run). For the Fmax confirmation
# run, Tmin = 10 - WNS is passed in via the environment variable STA_PERIOD.
if { [info exists ::env(STA_PERIOD)] } {
    set T $::env(STA_PERIOD)
} else {
    set T 10.0
}

# 1. Read the cell timing library
read_liberty $lib

# 2. Read and link the gate-level netlist
read_verilog synth/mult_array_netlist.v
link_design mult_array

# 3. Define the clock with period T
create_clock -name clk -period $T [get_ports clk]

# 4. Input arrival time (all inputs except the clock itself)
set_input_delay 1.0 -clock clk [delete_from_list [all_inputs] [get_ports clk]]

# 5. Output required time
set_output_delay 1.0 -clock clk [all_outputs]

# 6. Print the worst setup slack
report_worst_slack -max

# 7. Save the full critical path report
report_checks -path_delay max -digits 3 > sta/mult_array_timing.rpt

# Exit after the batch run (keeps `sta sta/sta.tcl` non-interactive)
exit
```

B.6 pnr/config.json

```
{
    "DESIGN_NAME": "mult_array",
    "VERILOG_FILES": [
        "dir::../rtl/full_adder.v",
        "dir::../rtl/mult_array.v"
    ],
    "CLOCK_PORT": "clk",
    "CLOCK_PERIOD": 7.0,
    "FP_SIZING": "absolute",
    "DIE_AREA": [
        0,
        0,
        150,
        150
    ]
}
```